Distributed Training of Large Language Models: Architectures, Parallelism, and Communication Challenges

Andrew Photinakis
Rochester Institute of Technology
acp7795@rit.edu
Rochester, New York, USA

I. Abstract

Training large language models beyond the memory and computational capacity of one accelerator requires distributing model state and computation across many devices. This comparative technical review examines the resulting trade-offs among data, tensor, and pipeline parallelism; state sharding; communication; and hardware topology. It then analyzes two recent systems: Mist, which co-optimizes parallelism, memory management, and overlap within GPU clusters, and PGCS, which overlaps expert-model computation with synchronization across wide-area networks. The comparison shows that memory savings often introduce additional communication and that performance depends on matching communication patterns to physical interconnects. It also identifies topology adaptation, safe multi-stream execution, and low-overhead failure recovery as important open problems.

II. Introduction

The memory required for model parameters, gradients, optimizer states, and activations makes large-language-model training infeasible on a single accelerator. Distributed training addresses this capacity limit by partitioning data, model operators, layers, or training state across multiple devices. Scaling out, however, does not produce proportional speedup: collective communication, pipeline bubbles, memory transfers, network contention, and stragglers can leave expensive accelerators idle [1], [2].

The central systems problem is therefore a three-way trade-off among per-device memory, communication cost, and computational utilization. Replicating state reduces communication complexity but exhausts device memory; sharding and offloading reduce memory consumption but introduce collectives or host–device transfers; and finer model partitioning increases the frequency of synchronization. The best configuration depends on both model structure and the hierarchy of links within and between nodes.

This paper first summarizes the parallelism and communication mechanisms needed to frame that trade-off. It then examines Mist's memory–parallelism co-optimization and PGCS's cross-data-center computation–synchronization pipeline. Finally, it compares their assumptions, considers scaling and reliability limits, and identifies open research directions for topology-aware distributed training.

III. Distributed Architectures & Algorithms

A. Memory, Data Parallelism, and State Sharding

For a model with Φ parameters, mixed-precision Adam training requires approximately 16Φ bytes for FP16 parameters and gradients, an FP32 master parameter copy, and two FP32 optimizer states; this estimate excludes activations, temporary workspaces, communication buffers, and fragmentation. Distributed Data Parallelism (DDP) replicates these states on every worker, whereas ZeRO and Fully Sharded Data Parallel (FSDP) progressively partition them across a data-parallel group. Table I compares their per-device memory and approximate ring-communication costs, where D is the data-parallel degree and $|P|$ and $|G|$ denote parameter and gradient bytes.

ZeRO progressively lowers memory consumption by sharding optimizer states, gradients, and eventually parameters. Full sharding in ZeRO-3 provides the largest capacity reduction, but it must reconstruct parameters through All-Gather operations during forward and backward execution. Assuming equal parameter and gradient sizes, ZeRO-3 communicates approximately 1.5× the per-device byte volume of DDP or ZeRO-2, trading memory capacity for additional communication.

B. Tensor Parallelism: Intra-Layer Model Parallelism

Tensor parallelism (TP), popularized by Megatron-LM, partitions individual Transformer-layer operations across T accelerators. It is commonly applied to the attention and multilayer perceptron (MLP) weight matrices. For an

TABLE I
PER-DEVICE COSTS OF DATA-PARALLEL STRATEGIES

Strategy	Sharded state	Static memory	Communication per iteration
DDP	None	16Φ	$2\frac{D-1}{D}\|G\|$
ZeRO-1	Optimizer states	$4\Phi + \frac{12\Phi}{D}$	$2\frac{D-1}{D}\|G\|$
ZeRO-2	Optimizer states and gradients	$2\Phi + \frac{14\Phi}{D}$	$\frac{D-1}{D}(\|G\| + \|P\|)$
ZeRO-3	Parameters, gradients, and optimizer states	$\frac{16\Phi}{D}$	$\frac{D-1}{D}(2\|P\| + \|G\|)$

MLP,

$$H = \mathrm{GeLU}(XW_1), \qquad Z = HW_2,$$

Megatron-style TP partitions W_1 by columns and W_2 by rows. Rank i first computes its activation partition and then a partial output:

$$H_i = \mathrm{GeLU}(XW_{1,i}), \qquad Z_i = H_i W_{2,i}, \qquad Z = \sum_{i=1}^{T} Z_i.$$

Because each row-parallel partition directly consumes its corresponding activation partition, the complete intermediate tensor H need not be reconstructed. An All-Reduce is nevertheless required to combine the partial outputs Z_i. Similar collectives occur in tensor-parallel attention, placing communication directly on the forward and backward critical paths.

TP reduces per-device memory and computation, but its frequent fine-grained collectives make it highly sensitive to latency and bandwidth. It is therefore typically confined to high-bandwidth intra-node interconnects such as NVLink or NVSwitch. Higher-dimensional TP variants can reduce per-device communication at larger scales, but require more complex partitioning, placement, and collective schedules.

C. Pipeline Parallelism: Inter-Layer Model Parallelism

Pipeline parallelism (PP) partitions a model along its layer dimension into p pipeline stages. Each stage contains a contiguous group of layers and is assigned to a different accelerator or compute node. A training batch is divided into m microbatches, which move through the stages as a pipeline. After a stage finishes processing one microbatch, it can begin processing the next while downstream stages continue working on earlier microbatches.

Adjacent stages exchange activation tensors during the forward pass and activation gradients during the backward pass. Pipeline parallelism therefore reduces the number of model parameters stored by each stage, but introduces point-to-point communication, pipeline bubbles, activation-storage requirements, and potential load imbalance between stages.

1) Pipeline Execution Schedules

GPipe fill-drain schedule and One-forward-one-backward schedule are two common pipeline schedules are the fill-drain and one-forward-one-backward schedules.

During GPipe fill-drain schedule, all m microbatches execute their forward passes through the pipeline. Backpropagation begins only after the final microbatch completes its forward pass. The pipeline then drains as the backward passes execute in reverse order. This schedule is synchronous and avoids weight staleness, but the forward phase creates a large number of outstanding activations. Activation checkpointing can reduce this memory requirement by retaining selected activations and recomputing others during the backward pass.

In one-forward-one-backward (1F1B) schedule interleaves forward and backward computations after an initial warm-up period. In steady state, a stage alternates between processing one forward microbatch and one backward microbatch. PipeDream popularized this form of interleaved scheduling, although different systems use different synchronization and weight-version policies. Compared with fill-drain scheduling, 1F1B limits the number of outstanding activations to a quantity proportional to the pipeline depth. Some asynchronous implementations may introduce weight staleness, whereas flush-based implementations preserve synchronous training semantics.

Assuming balanced stages and negligible communication overhead, processing m microbatches across p pipeline stages produces the bubble fraction

$$f_{\text{bubble}} = \frac{p-1}{m+p-1}.$$

Increasing the microbatch count reduces this overhead, although stage imbalance, communication delays, and activation-memory pressure can lower practical utilization.

D. Hybrid Parallelism and Specialized Extensions

Large-scale training systems commonly combine data, tensor, and pipeline parallelism into a three-dimensional parallel configuration. Within this organization, tensor-parallel groups divide individual layer operations, pipeline-parallel groups divide the model into layer stages, and data-parallel groups process different training samples. The configuration must be selected with respect to model dimensions, memory capacity, network topology, batch size, and communication cost. Increasing one parallel dimension can reduce a particular resource constraint while increasing communication or scheduling overhead elsewhere.

1) Sequence Parallelism

Sequence parallelism (SP) partitions activations along the input-sequence dimension across the members of a tensor-parallel group. It is particularly useful for operations such as layer normalization and dropout, which are otherwise replicated across tensor-parallel ranks.

Let X be an activation tensor with sequence length s. Under sequence parallelism, X is partitioned as

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_{d_{\text{TP}}} \end{bmatrix}, \qquad X_i \in \mathbb{R}^{(s/d_{\text{TP}})\times h}, \tag{1}$$

where h is the hidden dimension. Each rank stores and processes only its local sequence partition. Implementations can replace selected tensor-parallel All-Reduce operations with paired Reduce-Scatter and All-Gather operations. This preserves a comparable aggregate communication volume while reducing the peak activation memory stored by each rank. Sequence parallelism is therefore useful for models trained with long input sequences.

2) Expert Parallelism

Expert parallelism (EP) is designed for sparse mixture-of-experts (MoE) architectures. Instead of replicating every expert on every device, the E expert subnetworks are distributed across an expert-parallel group. A learned router assigns each input token to one or more experts.

If expert e is stored on a remote device, tokens assigned to that expert must be transferred to the owning device. This dispatch is commonly implemented using an All-to-All collective. After the selected experts process the tokens, a second All-to-All operation returns the results to their original ranks. Expert parallelism reduces the memory and computation required per device, but its performance depends on routing balance and All-to-All network performance. Uneven token assignments can create expert hot spots, communication imbalance, and accelerator idle time.

E. Communication Operations and Hardware Interconnects

Distributed-training performance depends on matching each communication pattern to the physical network. Tensor parallelism requires fast intra-node links such as NVLink or NVSwitch because it performs frequent layer-level collectives. Pipeline parallelism uses point-to-point transfers between stages, while DDP, ZeRO, and FSDP rely on inter-node collectives over fabrics such as InfiniBand or RoCE. CPU offloading uses the slower PCIe bus and can stall GPU execution unless transfers are prefetched and overlapped with computation.

Topology-aware placement therefore maps communication-intensive tensor parallelism to the fastest local links, pipeline stages to well-connected neighboring nodes, and data-parallel groups across the wider cluster. The following section examines systems that jointly optimize these communication, memory, and scheduling decisions.

IV. Research Advances & Synthesis

Emerging research systems move beyond static, manually tuned parallelization topologies toward dynamic, overlap-aware, and hardware topology-optimized execution frameworks. These advanced systems automatically explore non-trivial execution schedules, hide memory movement latency, and compensate for dynamic network bottlenecks.

A. System Analysis 1: Mist

The Mist framework [3] introduces memory, overlap, and imbalance-aware automatic co-optimization for LLM distributed training. Prior auto-parallelism planners (e.g. Alpa, Aceso, AdaPipe) tuned parallelism grids while keeping memory configurations fixed, or limited their optimization to single techniques such as static activation recomputation. Consequently, baseline planners suffered from mulltiple severe failure modes. Alpa failed to find feasible solutions on memory-constrained L4 GPUs due to its memory-unaware intra-operator pass causing OOM errors. Aceso underperformed Megatron-LM in 6 out of 10 evaluation cases because it lacked sharded data parallelism and communication-computation overlap awareness.

1) Fine-Grained Four-Stream Overlap Scheduling

Mist introduces a fine-grained four-stream overlap schedule that coordinates computation, collective communication, and CPU–GPU data transfers. The objective is to hide communication and swapping latency behind independent GPU computation, thereby reducing stalls on the critical execution path. Table II summarizes the role of each CUDA stream during the forward and backward passes.

TABLE II
Mist four-stream overlap schedule

CUDA stream	Forward-pass operation	Backward-pass operation
Stream 1: Compute	Forward computation F_i	Backward computation B_i
Stream 2: NCCL	Parameter All-Gather for P_{i+1}	Gradient Reduce-Scatter for R_{i-1}
Stream 3: D2H	Activation offload, $A_i \rightarrow$ host memory	Gradient offload, $G_i \rightarrow$ host memory
Stream 4: H2D	Parameter swap-in, $P_{i+1} \leftarrow$ host memory	Activation swap-in, $A_i \leftarrow$ host memory

The compute stream executes the forward or backward kernels for the current layer or pipeline stage. Concurrently, the NCCL stream performs collective communication for neighboring operations. The device-to-host (D2H) stream offloads tensors that are not immediately required, while the host-to-device (H2D) stream prefetches tensors before their next use. Effective overlap requires transfers to begin early enough to complete before the corresponding tensor becomes a dependency of the compute stream.

Mist also controls the fraction of each model-state category that is offloaded. Let

$$O_P,\ O_G,\ O_O,\ O_A \in [0, 1] \tag{2}$$

denote the offloading ratios for parameters, gradients, optimizer states, and activations, respectively. These controls can be represented as the vector

$$\mathbf{O} = \begin{bmatrix} O_P & O_G & O_O & O_A \end{bmatrix} \in [0, 1]^4. \tag{3}$$

For a state category X with total size M_X, an offloading ratio O_X corresponds to

$$M_X^{\text{host}} = O_X M_X, \qquad M_X^{\text{device}} = (1 - O_X) M_X. \tag{4}$$

Thus, $O_X = 0$ retains the entire state category in GPU memory, whereas $O_X = 1$ offloads the entire category to host memory when it is not required for computation. Intermediate ratios allow Mist to trade GPU-memory savings against PCIe transfer overhead and the available opportunities for overlapping those transfers with computation.

To evaluate millions of parallel combinations without executing concrete graph simulations, Mist employs a Symbolic-Based Efficient Performance Analysis System:

1) Symbolic Shape Execution & Tracing: Inputs, model dimensions, and parallel degrees are declared as abstract symbolic expressions (b, s, h, d, TP, PP, DP). Mist executes a single symbolic simulation pass using PyTorch fake tensors and meta-devices to build a symbolic computation graph.
2) Batched Value Substitutions: Analytical runtime and peak memory usage are derived as symbolic parametric equations. Candidate plans are evaluated via low-latency symbolic matrix substitutions rather than concrete operator executions, accelerating search space exploration by over $10^5\times$.
3) Data-Driven Multi-Op Interference Model: Accurately quantifies performance degradation when compute kernels, NCCL collectives, D2H transfers, and H2D transfers execute concurrently over shared buses.

Empirical telemetry verifies that Mist's symbolic performance analyzer achieves outstanding accuracy, recording an average runtime error ratio of 1.79% and an average memory footprint error ratio of 2.10%.

To address execution imbalance across pipeline microbatches, Mist uses imbalance-aware hierarchical tuning. Intra-stage tuning searches a Pareto frontier between steady-state microbatch latency h_x and the first- or last-microbatch deviation i_x introduced by parameter gathering, gradient reduction, offloading, and prefetching. Inter-stage tuning then uses a mixed-integer linear program to select pipeline partitions, activation recomputation, and rank assignments.

Its objective combines steady-state execution, pipeline fill and drain, and microbatch imbalance:

$$\min \mathcal{L} = (m-1)\max_x h_x + \sum_{x=1}^{p} h_x + \max_x \left(i_x - \sum_{u=1}^{x-1} h_u \right),$$

where p is the number of pipeline stages and m is the number of microbatches. This formulation prevents tuning decisions based only on steady-state execution from overlooking the additional costs of boundary microbatches.

2) Cumulative Performance Improvements

Mist's performance improvement results from progressively combining flexible activation checkpointing, fine-grained CPU offloading, and imbalance-aware pipeline scheduling.

Across the evaluated GPU clusters, Mist achieves an average speedup of **1.28**×, with a maximum speedup of **1.73**×, relative to Megatron-LM. Compared with Aceso, Mist achieves an average speedup of **1.27**× and a maximum speedup of **2.04**× [3].

At a larger evaluated scale, consisting of an 80-layer GPT-3 model trained on 32 NVIDIA L4 GPUs, Mist achieves a speedup of up to **1.32**×. The relative benefit is greatest in memory-constrained and PCIe-constrained L4 environments, where fine-grained offloading and computation–communication overlap can hide part of the host–device transfer cost. In higher-bandwidth A100 environments connected through NVLink and InfiniBand, the baseline already operates closer to the available hardware limits. Consequently, Mist has fewer remaining communication and data-transfer stalls to eliminate, producing smaller relative speedups.

B. System Analysis II: PGCS

Parallel Gradient Computation and Synchronization (PGCS) targets the distributed training of expert language models across geographically separated data centers. In these environments, limited bandwidth, long propagation delays, and heterogeneous connectivity can make global gradient synchronization a dominant component of training time [4].

PGCS addresses this problem through three mechanisms. First, each data center uses worker groups for gradient computation and parameter servers for aggregation, model updates, and parameter distribution. Synchronization is hierarchical: gradients are aggregated locally before parameter servers synchronize across data centers. Worker groups may combine tensor parallelism within hosts and pipeline parallelism between hosts.

Second, PGCS alternates expert-model execution so that synchronization for one expert overlaps gradient computation for the next:

$$\text{Sync}(E_j) \parallel \text{Compute}(E_{j+1}).$$

The experts share a common architecture, allowing workers to switch parameter values without loading a different computational structure. Local batch sizes are also adjusted to the computing and communication capacity of each site.

Third, PGCS uses a bisection and linear-programming algorithm (BLP) to minimize global synchronization latency. Bisection searches for the lowest feasible latency, while the linear program assigns gradient partitions, communication paths, and link bandwidth subject to topology, capacity, and propagation-delay constraints.

Together, these mechanisms reduce accelerator idle time by hiding WAN communication behind expert computation. However, PGCS assumes a predefined expert order, structurally uniform experts, and sufficiently stable network information. Dynamic congestion, routing changes, and link failures therefore remain important limitations.

C. System Analysis III: Production-Scale Systems and Survey Synthesis

Recent surveys organize the requirements of distributed LLM training around three system properties: scalability, efficiency, and reliability (SER) [1]. These properties are interdependent. Increasing cluster size can expose communication bottlenecks and raise the probability of component failures, while techniques that improve memory efficiency can introduce additional communication or recomputation.

TABLE III
SCALABILITY, EFFICIENCY, AND RELIABILITY FRAMEWORK

Property	**System objective**	**Representative techniques**
Scalability	Expand training to large accelerator clusters while preserving correctness and useful scaling efficiency.	Hybrid parallelism, ZeRO/FSDP sharding, expert parallelism, and topology-aware placement.
Efficiency	Maximize accelerator utilization while controlling computation, communication, and memory overhead.	Communication–computation overlap, kernel fusion, optimized attention, memory offloading, and collective scheduling.
Reliability	Detect, isolate, and recover from hardware, network, and software failures without substantial loss of training progress.	Asynchronous checkpointing, anomaly detection, fault isolation, in-memory recovery, and checkpoint-free recovery.

1) Representative Efficiency Techniques

The survey literature identifies several systems that address different components of the SER framework: CO2, EDiT, & EST [2].

CO2 seeks to overlap the forward pass, backward pass, gradient All-Reduce, and optimizer update. It uses a dual-buffer pipeline to execute the computation of one microbatch concurrently with communication associated with another microbatch. Separate CUDA and NCCL streams provide nonblocking execution. Although this design can reduce exposed communication latency, highly asynchronous execution can introduce race conditions if tensor dependencies are not enforced correctly.

EDiT is based on local stochastic gradient descent. It performs local updates within workers and uses hierarchical gradient aggregation for synchronization across workers. Its adaptive policy changes the synchronization interval according to observed gradient variance. Longer local-update intervals reduce communication frequency but may increase divergence between local models and adversely affect convergence.

EST uses a dynamic subnet-evolution process. During an exploration phase, it activates different parameter subsets and estimates their importance from gradient information. During the utilization phase, important parameters are retained while other active subnet parameters are rotated. By maintaining optimizer state only for active parameters and communicating gradients only for the active subnet, EST reports a 70% reduction in memory consumption and a 10× reduction in communication volume under its evaluated configurations.

2) Reliability at Production Scale

Production-scale training jobs may occupy thousands of accelerators for weeks or months. At this scale, failures are expected during a training run rather than treated as exceptional events. Reported failure sources include GPU errors, ECC errors, NVLink failures, NCCL timeouts, network connection errors, software failures, power interruptions, and cooling problems [1].

The infrastructure survey reports that hardware problems accounted for 78% of the failures observed during LLaMA 3 pretraining. It also reports that Alibaba's C4 analysis found approximately 82.5% of errors to be confined to a specific node or device. Even when a fault does not terminate the training job, link degradation and computational stragglers can lower Model FLOPs Utilization (MFU) because synchronous workers must wait for the slowest participant.

Traditional synchronous checkpointing pauses training while model state is copied and persisted to storage. As model size and accelerator count increase, this pause can become a substantial source of lost training time. Asynchronous checkpointing instead overlaps snapshot and persistence operations with subsequent training computations. A conceptual asynchronous checkpoint pipeline can be written as

$$\text{Train}(t+1) \parallel \text{Snapshot}(t) \parallel \text{Persist}(t-1), \tag{5}$$

where the current training iteration proceeds while recent state is copied to host memory and an earlier snapshot is written to persistent storage. This approach reduces checkpoint stalls but requires careful consistency control to ensure that a recoverable snapshot corresponds to a valid training state.

The SER framework therefore emphasizes that scalability cannot be evaluated only through peak throughput. A production training system must also maintain high utilization during failures, recover without excessive rollback, and prevent communication or storage operations from repeatedly blocking the training critical path.

V. Scope, Critique & Challenges

No single distributed training framework eliminates all computational, memory, and network bottlenecks simultaneously. High-performance software design rely on key structural assumptions that introduce distinct trade-offs under real-world production conditions.

A. Rigorous System Critique and Architectural Limitations

1) Critique of Mist (EuroSys '25)

Mist's intra-stage performance model assumes identical Transformer layer execution blocks within a pipeline stage. While valid for standardized decoder architectures (e.g. GPT-3, LLaMa), this assumptions breaks down on non-uniform models, such as Mixture-of-Experts (MoE) architectures with sparse expert layer placements, or hybrid models combining state-space matrix (e.g. Mamba) with attention blocks. Furthermore, when aggressive host memory offloading ratios (O_P, O_G, O_O, O_A) are selected on PCIe-based clusters (e.g., NVIDIA L4 nodes), the concurrent execution of Stream 3 (D2H) and Stream 4 (H2D) rapidly saturates bidirectional PCIe bus bandwidth. PCIe Interconnect & Host Bus Contention Mode Concurrent activation swap-out (Stream 3 D2H) and parameter swap-in (Stream 4 H2D) create heavy memory traffic across host CPU DRAM channels. This contention induces NUMA node memory thrashing and CPU cache line invalidation, causing execution latency spikes that invalidate Mist's analytical interference model predictions.

2) Critique of PGCS (IEEE TNSE)

PGCS relies on a static alternating expert computation schedule optimized using Bisection Linear Programming (BLP) network flow solutions. On production cross-site WAN links, dynamic IP routing changes, transient packet loss, and bursty background traffic introduce unexpected network jitter. Because PGCS lacks runtime dynamic re-pipelining, a single transient bandwidth drop on one WAN link can stall all parameter servers, destabilizing synchronous gradient updates across nodes.

B. System Scalability and Interconnect Saturation Barriers

At large cluster scales, ZeRO-3 parameter All-Gathers and mixture-of-experts All-to-All exchanges generate substantial cross-rack traffic. Insufficient bisection bandwidth forces collective flows to compete for shared links, increasing queueing, retransmissions, and synchronization latency. More connected network topologies alleviate this bottleneck but increase hardware, power, and placement complexity.

Synchronous training also amplifies stragglers because each step is determined by its slowest rank:

$$t_{\text{step}} = \max_{1 \le i \le N} t_i.$$

Network congestion, thermal throttling, operating-system interference, or degraded hardware on one accelerator can therefore delay the entire training group. As cluster size increases, straggler detection and mitigation become essential to maintaining utilization.

Memory reduction creates a related communication trade-off. ZeRO-3 and FSDP reduce per-device state by sharding parameters, but must reconstruct them through All-Gather operations during forward and backward execution. Assuming equal parameter and gradient sizes, ZeRO-3 communicates approximately 1.5× the per-device byte volume of DDP or ZeRO-2. Effective scaling must therefore jointly optimize sharding, microbatch size, parallelism, and topology rather than minimizing memory in isolation.

VI. Conclusion

Co-optimizing memory footprints, communication latencies, and system reliability is essential for scaling Large Language Models past single-accelerator capacity bounds. Modern dynamic execution engines demonstrate that co-designing software scheduling with physical interconnect topologies unlocks significant performance gains over static baseline frameworks. By orchestrating fine-grained multi-stream overlap, state sharding, and symbolic performance modeling, architectures such as Mist achieve substantial throughput speedups while preserving strict computational fidelity. However, physical bandwidth ceilings, network jitter, and straggler amplification at extreme cluster scales present persistent infrastructure bottlenecks. Resolving these challenges will require next-generation distributed systems to integrate elastic failure recovery, adaptive cross-site routing, and formally verified concurrent execution streams.

References

[1] J. Duan, S. Zhang, Z. Wang, L. Jiang, W. Qu, Q. Hu, G. Wang, Q. Weng, H. Yan, X. Zhang, X. Qiu, D. Lin, Y. Wen, X. Jin, T. Zhang, and P. Sun, "Efficient training of large language models on distributed infrastructures: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2407.20018

[2] F. Zeng, W. Gan, Y. Wang, and P. S. Yu, "Distributed training of large language models: A survey," *Natural Language Processing Journal*, vol. 12, p. 100174, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2949719125000500

[3] Z. Zhu, C. Giannoula, M. Andoorveedu, Q. Su, K. Mangalam, B. Zheng, and G. Pekhimenko, "Mist: Efficient distributed training of large language models via memory-parallelism co-optimization," in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys '25. ACM, Mar. 2025, pp. 1298–1316. [Online]. Available: http://dx.doi.org/10.1145/3689031.3717461

[4] H. Li, H. Jiang, J. Wu, G. Yang, and J. Zhang, "Parallel gradient computation and synchronization: Enhancing the efficiency of distributed training for llms," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 1962–1976, 2026.